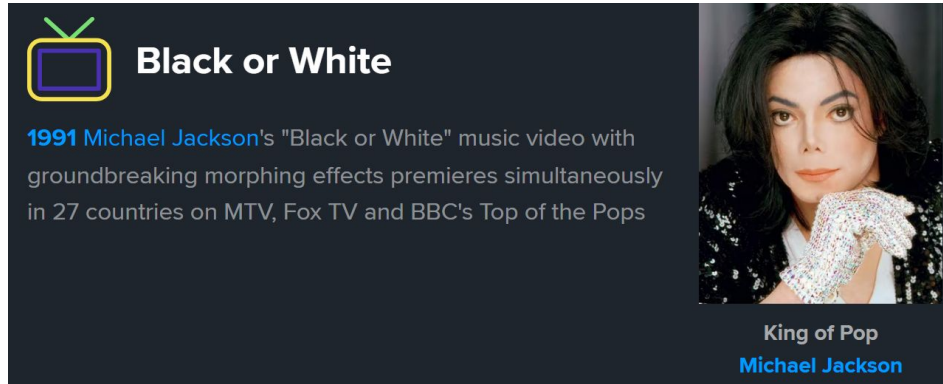


COMS W4156 Advanced Software Engineering (ASE)

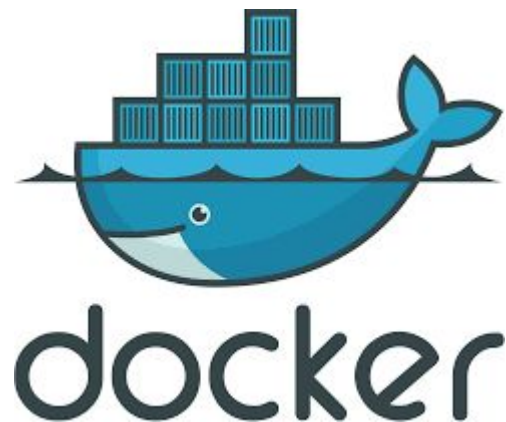
November 14, 2024

On this day ...



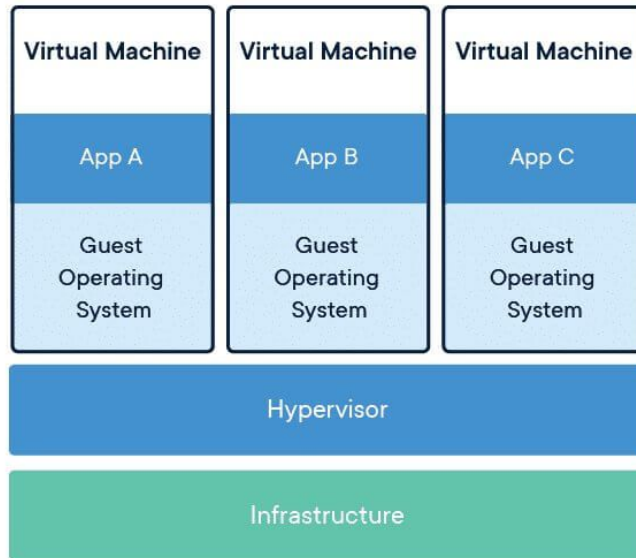
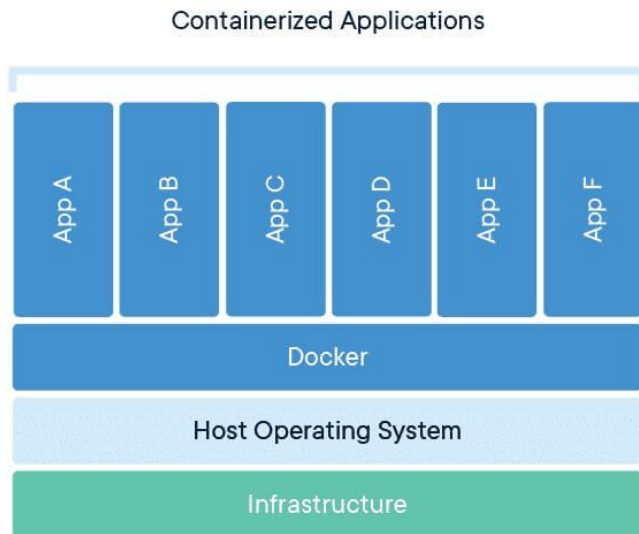
Agenda

1. Docker demo: Justin Tan
2. Origin of Design Patterns
3. Singleton
4. Adapter
5. Observer
6. Deadlines



Justin Tan Docker Demo

[demo slides](#)



Agenda

1. Docker demo: Justin Tan
2. Origin of Design Patterns
3. Singleton
4. Adapter
5. Observer
6. Deadlines



How do we choose a design for a particular kind of software development problem?

We look at what everyone else uses for that kind of problem or similar problems

- *Originality is overrated*
- *Imitation is the sincerest form of not being stupid*



Our (not yet written) software almost certainly corresponds to some “pattern”

Architectural Patterns



Before there were patterns in software, before there was any software, there were patterns in cities and individual buildings

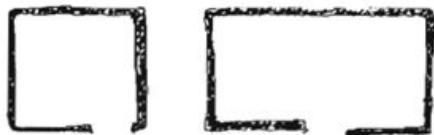
Christopher Alexander codified what architects and civil engineers had long known in his books about patterns

On building architecture: *“Each pattern describes a problem which occurs over and over again in our environment, and then describes the core of the solution to that problem, in such a way that you can use this solution a million times over, without ever doing it the same way twice.”*



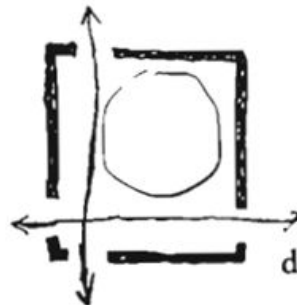
The success of a room depends to a great extent on the position of the doors. If the doors create a pattern of movement which destroys the places in the room, the room will never allow people to be comfortable.

First there is the case of a room with a single door. In general, it is best if this door is in a corner. When it is in the middle of a wall, it almost always creates a pattern of movement which breaks the room in two, destroys the center, and leaves no single area which is large enough to use. The one common exception to this rule is the case of a room which is rather long and narrow. In this case it makes good sense to enter from the middle of one of the long sides, since this creates two areas, both roughly square, and therefore large enough to be useful. This kind of central door is especially useful when the room has two partly separate functions, which fall naturally into its two halves.



Rooms with one door.

corners



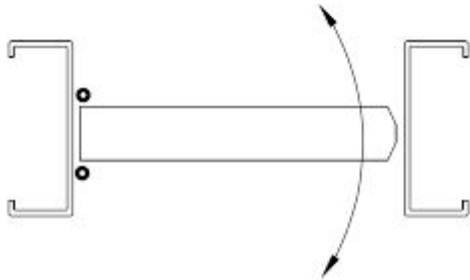
doorways



Except in very large rooms, a door only rarely makes sense in the middle of a wall. It does in an entrance room, for instance, because this room gets its character essentially from the door. But in most rooms, especially small ones, put the doors as near the corners of the room as possible. If the room has two doors, and people move through it, keep both doors at one end of the room.

If we were designing buildings instead of software, would Professor Alexander's pattern solve our problem of where to put doors?

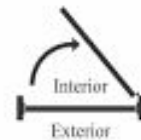
Not entirely - which way should the door(s) swing?



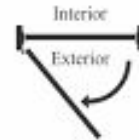
Swing Door Handing



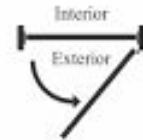
Left Hand
Opens To Interior



Right Hand
Opens To Interior

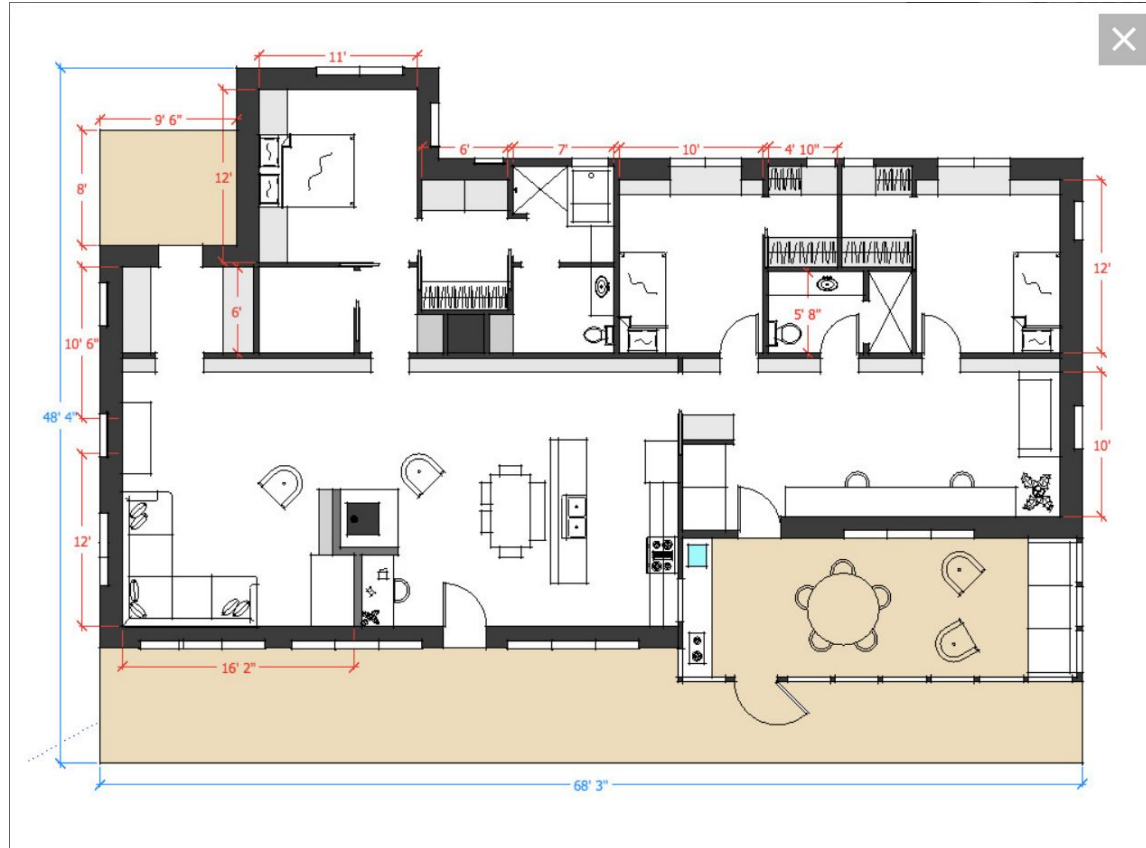


Left Hand Reverse
Opens To Exterior



Right Hand Reverse
Opens To Exterior

A small house attempting to use Alexander's "A Pattern Language"



Architecture patterns do not provide solutions to every aspect of the problem



paraveina

6 years ago

Speaking from experience, that door to the porch will be slammed against your WD every time it opens. I suggest it should either open out, or if you still want to be able to do a screen, swing the other way or move it down slightly.

👍 Like ❤️ Save

chelwa thanked paraveina



Alexander's Architectural Pattern Elements

1. Name of the pattern
2. Purpose of the pattern = what problem it solves
3. How to use the pattern to solve the problem
4. Constraints to consider in solution

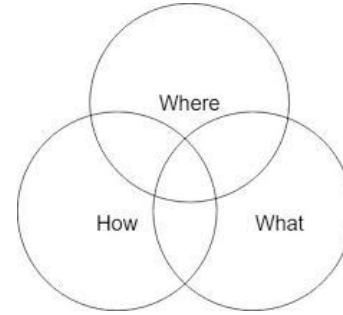


These elements work just as well for software as for buildings

- Reuse well-understood and tested architectures and designs
- Save time and don't reinvent the wheel
- Communication language among software engineers

What is the Difference between (Software) Architecture and Design?

Software engineers need version control (like [git](#)) because software changes over time, possibly very many times over very long periods of time



Non-trivial software is not implemented all-at-once, and software with real users changes to fix bugs, add features, and adapt to changing environment and context. Refactorings along the way may substantially re-organize system structure

Design principles. such as the [SOLID principles](#), are centered on change, trying to make the code more resilient to change - more flexible, extensible, and maintainable (micro level)

[Architecture](#) is fundamental system structure that rarely changes (macro level) - client/server, N-tier, broker, pub/sub, etc. (exception: monolithic -> SOA -> microservices)

Gang of Four book

(Software) [Design Patterns](#) are general reusable solutions to commonly occurring software development problems, typically defined by a description or template rather than a library or framework with reusable code

Initial set of 23 design patterns presented in GoF [book](#) (1994), heavily influenced by Christopher Alexander's [architectural patterns for buildings and cities](#) (1977)

GoF book gives examples in C++ and Smalltalk
([Java wasn't available yet](#))

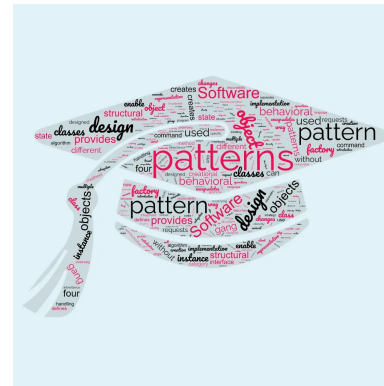
See “15 years later” interview with GoF authors [here](#)



Behavioral vs. Structural vs. Creational

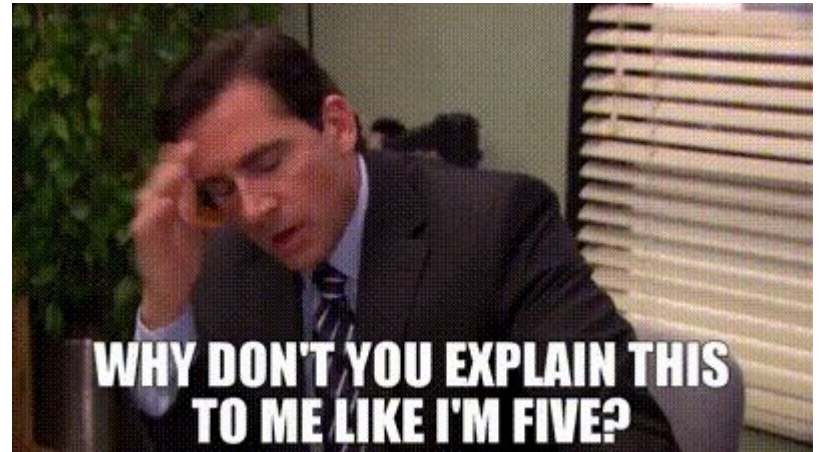
- [Creational patterns](#) provide class instantiation and object creation mechanisms that increase flexibility and reuse of existing code
- [Structural patterns](#) explain how to assemble objects and classes into larger structures, while keeping the structures flexible and efficient
- [Behavioral patterns](#) realize common communication paradigms and the assignment of responsibilities between objects

Discuss a few now as time permits, more next week



Agenda

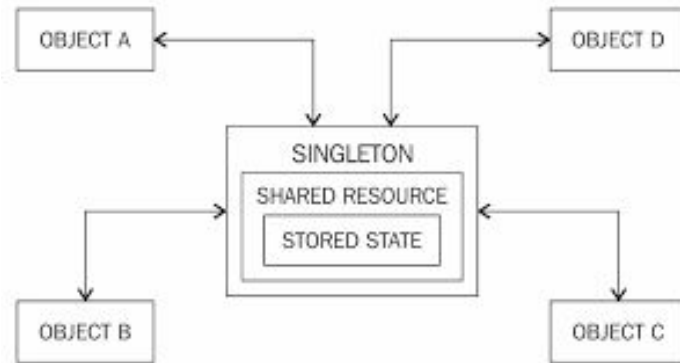
1. Docker demo: Justin Tan
2. Origin of Design Patterns
3. Singleton
4. Adapter
5. Observer
6. Deadlines



Singleton is one of the simplest design patterns

Singleton lets you ensure that a class has only one (or at most one) instance, while providing a global access point to this instance

Typically controls access to a shared resource such as database, cache, thread pool, logger, device





When to Use Singleton

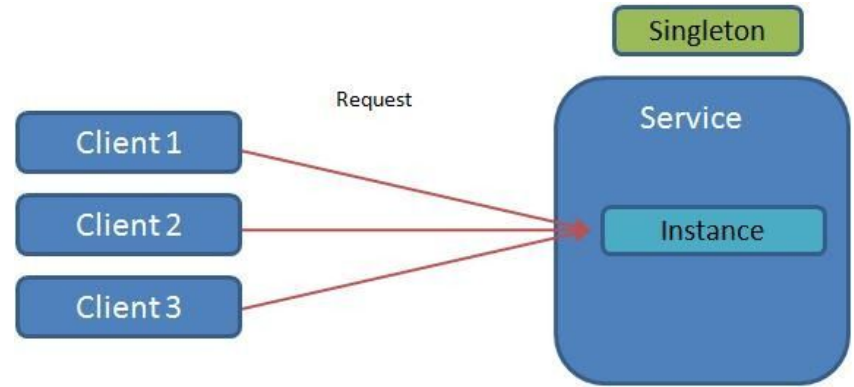
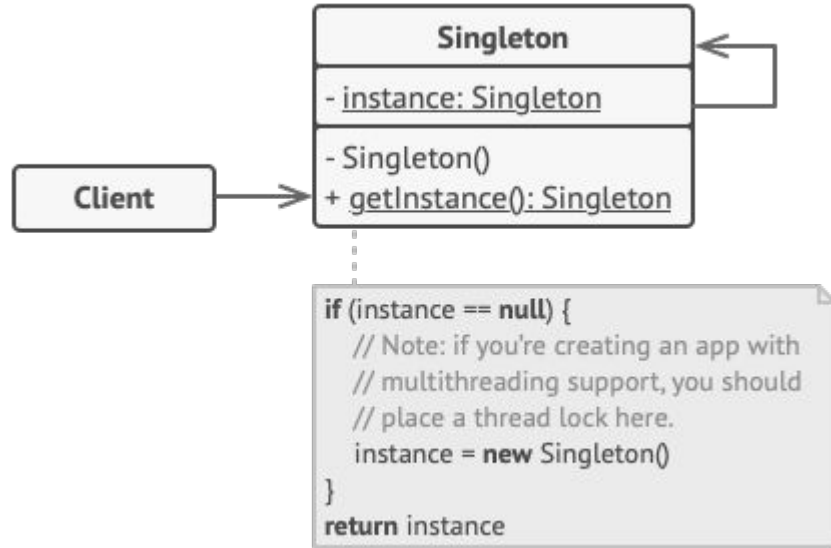
Use when a class in your program should have just a single instance available to the rest of the codebase; for example, a single database object shared by different parts of the program

The [Singleton pattern](#) disables all other means of creating objects of a given class except for the special creation method, which either creates a new object or returns an existing one if it has already been created

Singleton guarantees that there's just one (or at most one) instance and nothing, except for the Singleton class itself, can replace the cached instance

Can be generalized to situations where you want exactly N (or up to N)

Singleton and Thread Safety



Thread one

```
public static Singleton getInstance(){  
    if(obj==null)  
  
    obj=new Singleton();  
    return obj;  
}
```

Thread two

```
public static Singleton getInstance(){  
    if(obj==null)  
  
    obj=new Singleton();  
    return obj;  
}
```

Singleton Pattern Code Examples

C++

Java



Agenda

1. Docker demo: Justin Tan
2. Origin of Design Patterns
3. Singleton
4. **Adapter**
5. Observer
6. Deadlines

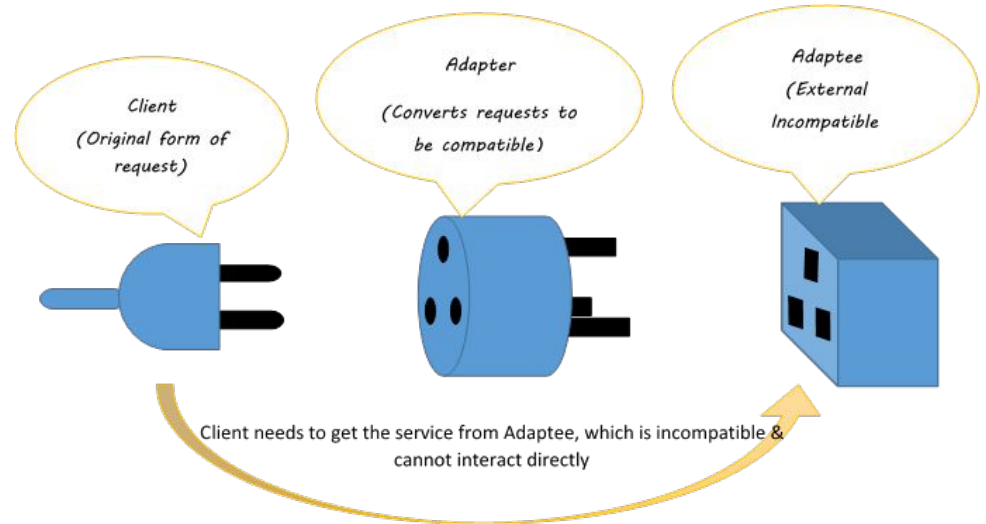
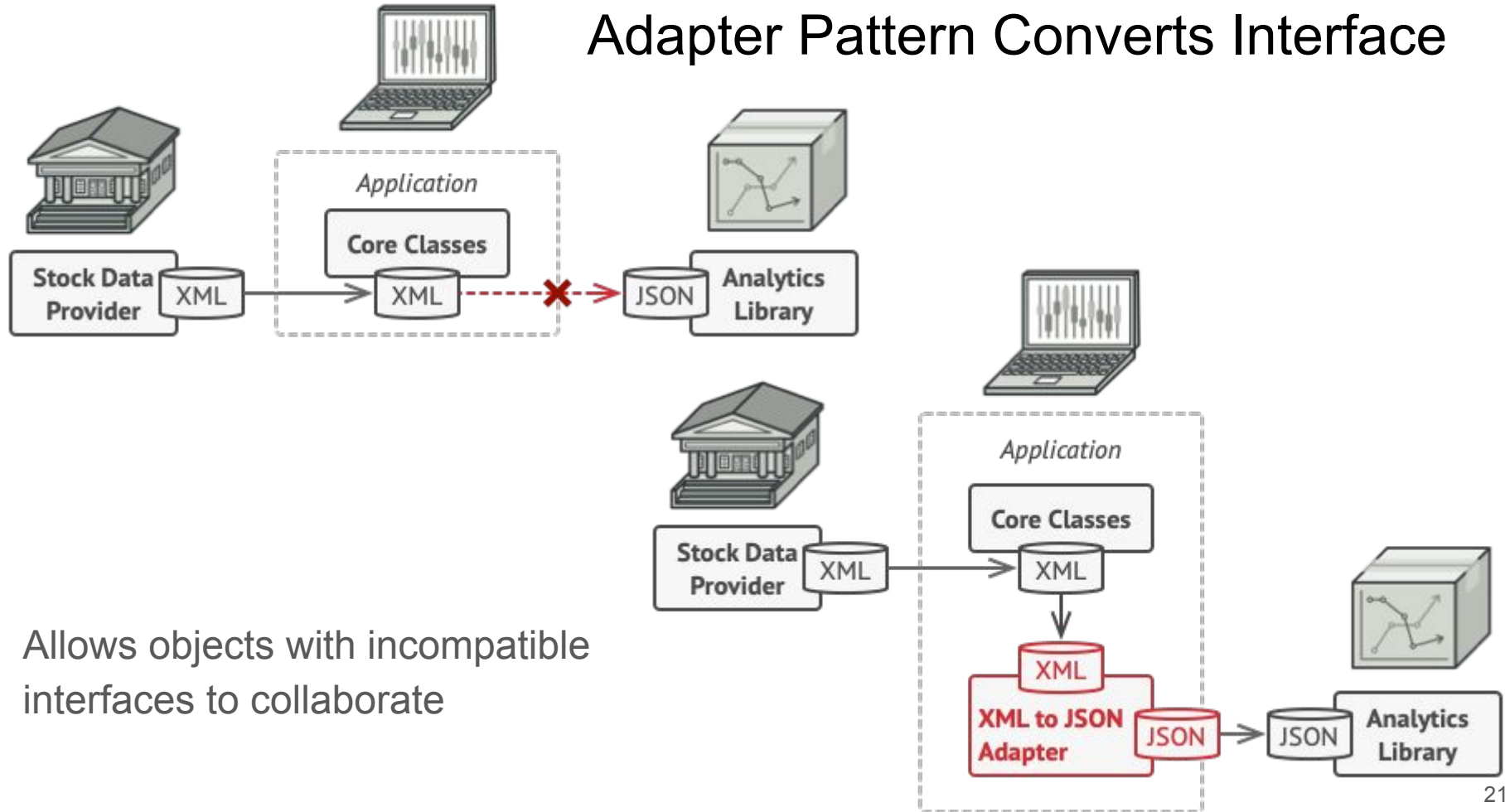


Figure 1-Adapter Pattern Concept

Adapter Pattern Converts Interface



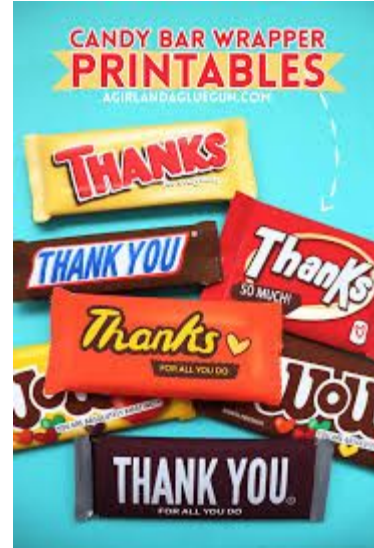
Allows objects with incompatible interfaces to collaborate

When to Use Adapter

Adapter ~= wrapper

Use when you want to use some existing component, but its interface isn't compatible with the rest of your code

The Adapter pattern creates a middle layer to translate API calls, and their inputs and outputs, between your code and legacy code, 3rd-party code or any other code that cannot be changed



Adapter Pattern Code Examples

[C++](#)

[Java](#)



Agenda

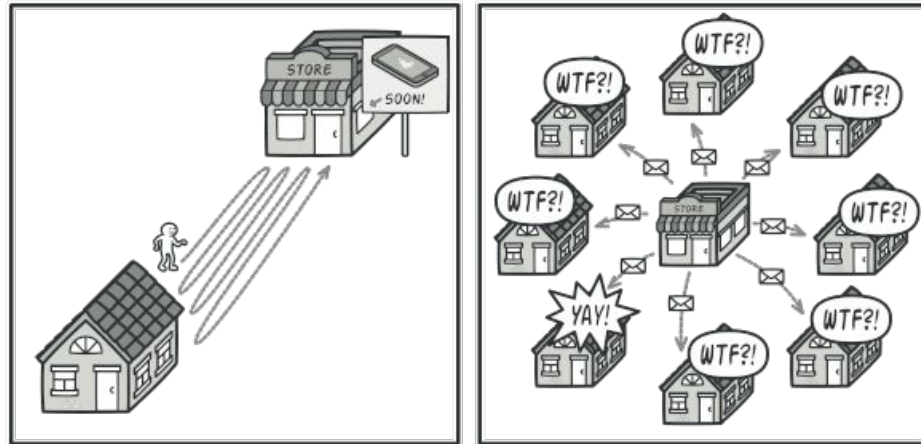
1. Docker demo: Justin Tan
2. Origin of Design Patterns
3. Singleton
4. Adapter
5. **Observer**
6. Deadlines



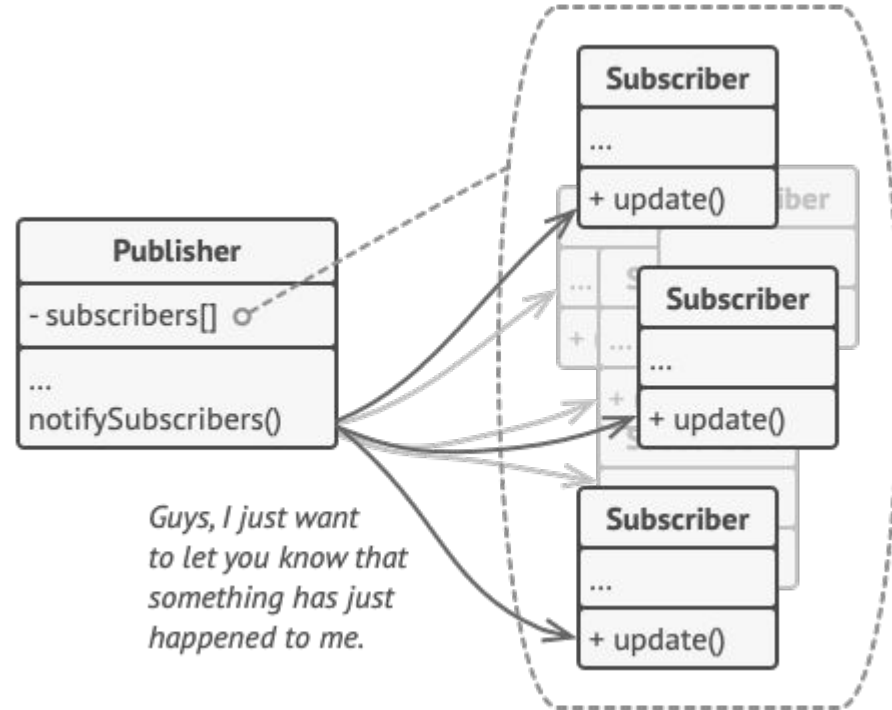
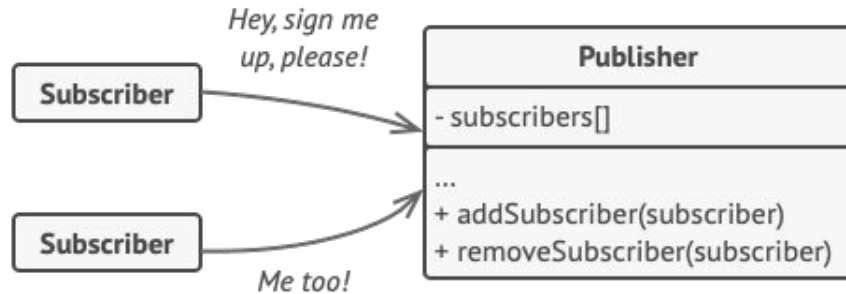
Observer Problem

Lets you define a subscription mechanism to notify multiple objects (called subscribers) about any events that happen to the object they're observing (called publisher or subject)

Analogous to [publish/subscribe architecture](#), but among objects instead of systems



Observer Solution



When to Use Observer



Use when changes to the state of one object may require changing other objects, particularly when the actual set of objects is unknown beforehand or changes dynamically

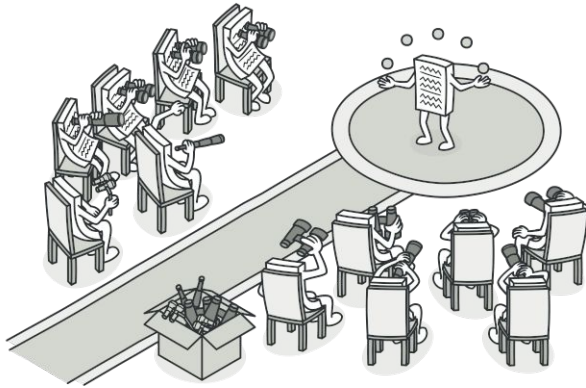
The Observer pattern lets any object that implements the subscriber interface subscribe for event notifications from publisher objects, e.g., to hook up custom code (callbacks) via custom subscriber classes

Use when some objects in your program must observe others, but only for a limited time or in specific cases - the subscription list is dynamic, so subscribers can join or leave the list whenever they need to

Observer Pattern Code Examples

C++

Java



Behavioral vs. Structural vs. Creational

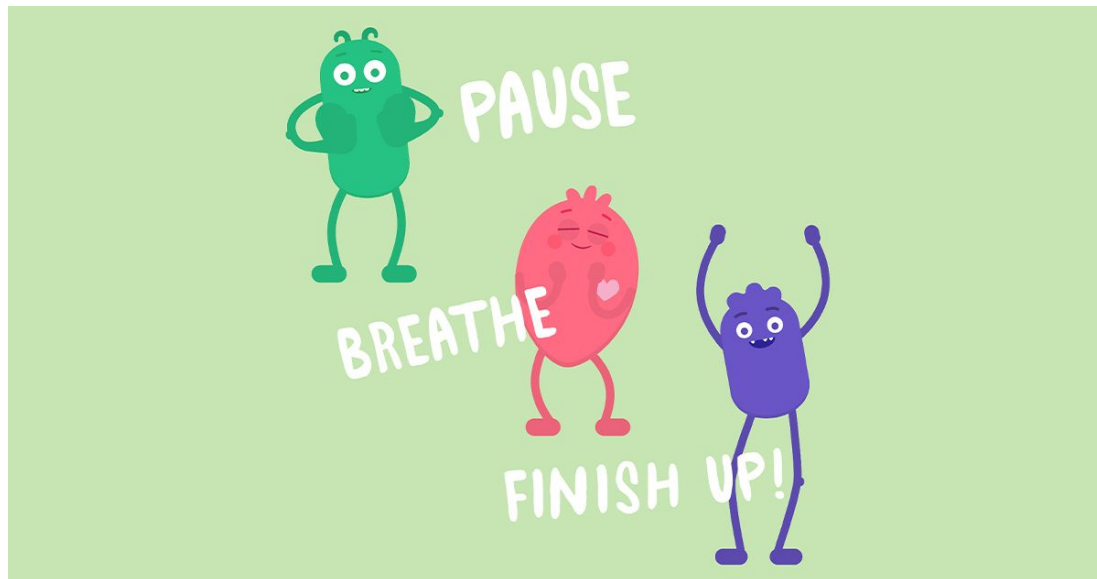
- [Creational patterns](#) provide class instantiation and object creation mechanisms that increase flexibility and reuse of existing code
- [Structural patterns](#) explain how to assemble objects and classes into larger structures, while keeping the structures flexible and efficient
- [Behavioral patterns](#) realize common communication paradigms and the assignment of responsibilities between objects

I mentioned one for each kind, which was which?



Agenda

1. Docker demo: Justin Tan
2. Origin of Design Patterns
3. Adapter
4. Observer
5. Singleton
6. **Deadlines**



Second Iteration due November 25
(Monday before Thanksgiving)

Second Iteration Demo due December 6
(Friday the week after Thanksgiving)

Second Assessment Friday December 6
12:01AM to Monday December 9
11:59PM (last day of classes and the day
before university "study days")

Demo Day Monday December 16 10AM to
4PM (Monday of finals week)



Next Week

More Design Patterns





Ask Me Anything